Fundamentals of Fog and Edge Computing

BCSE313L

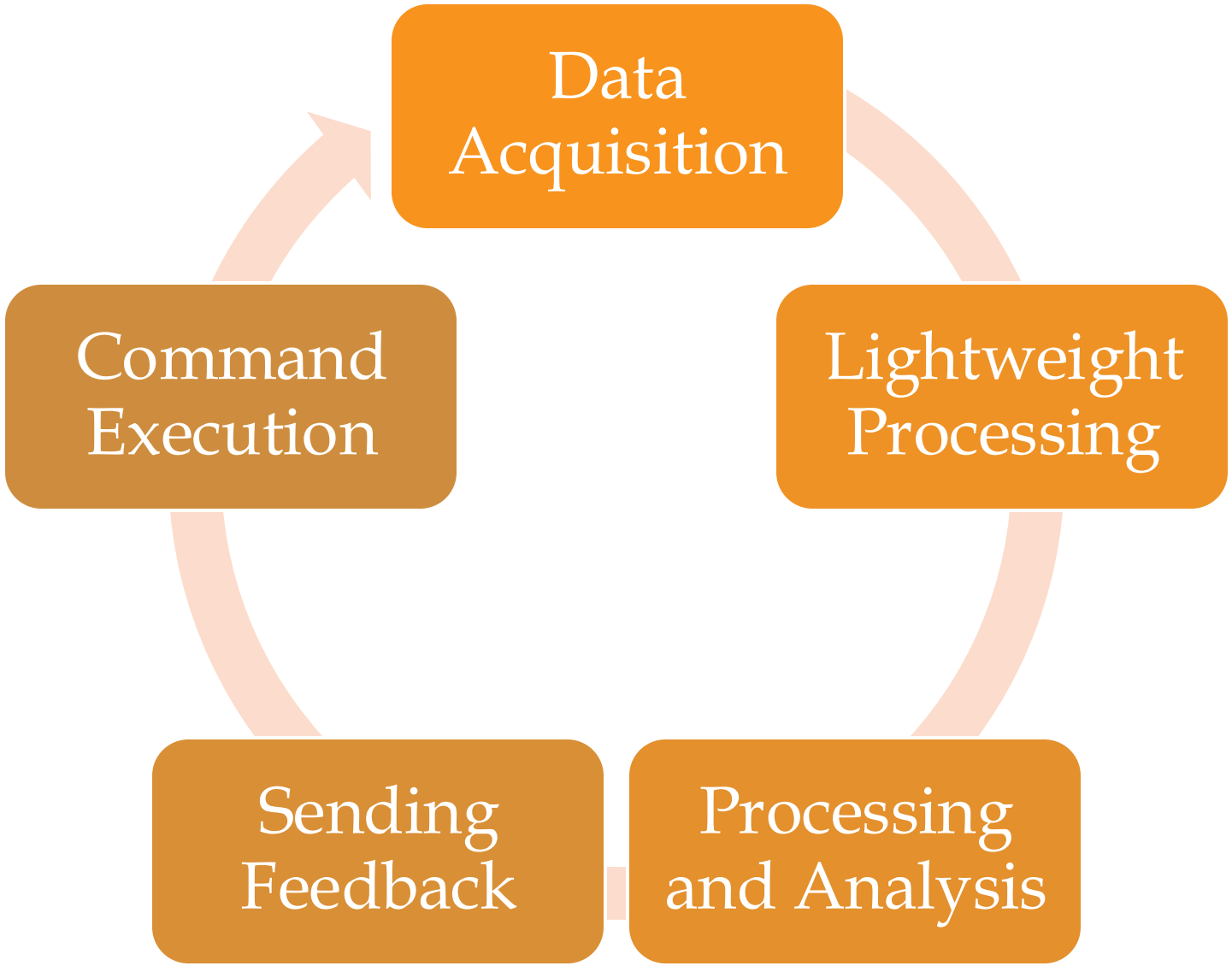
WINTER 2024 - 2025

Module 6 - Technologies in Fog Computing

Fog Data Management

- It focuses on handling the entire data life cycle efficiently while addressing the unique characteristics of IoT data.

Fog Data Management - Fog Data Life Cycle



Fog Data Management - Fog Data Life Cycle

Data Acquisition

- Data is collected from end devices (e.g., sensors, actuators).
- Data can be sent directly to the fog or via a sink node/local gateway.

Lightweight Processing

- Involves lightweight data manipulation such as aggregation, filtering, cleaning, compression /decompression, and pattern extraction.
- Provides pre-processing feasibility by leveraging locally stored data in fog devices.
- Feedback data may require decompression, decryption, or format changes before being sent back to the device layer.

Fog Data Management - Fog Data Life Cycle

Processing and Analysis

1. Data is permanently stored in the cloud layer for deeper analysis.
2. Big data platforms (e.g., HDFS, MapReduce) are used for large-scale processing and analytics.
3. Users can access reports or insights derived from stored data.

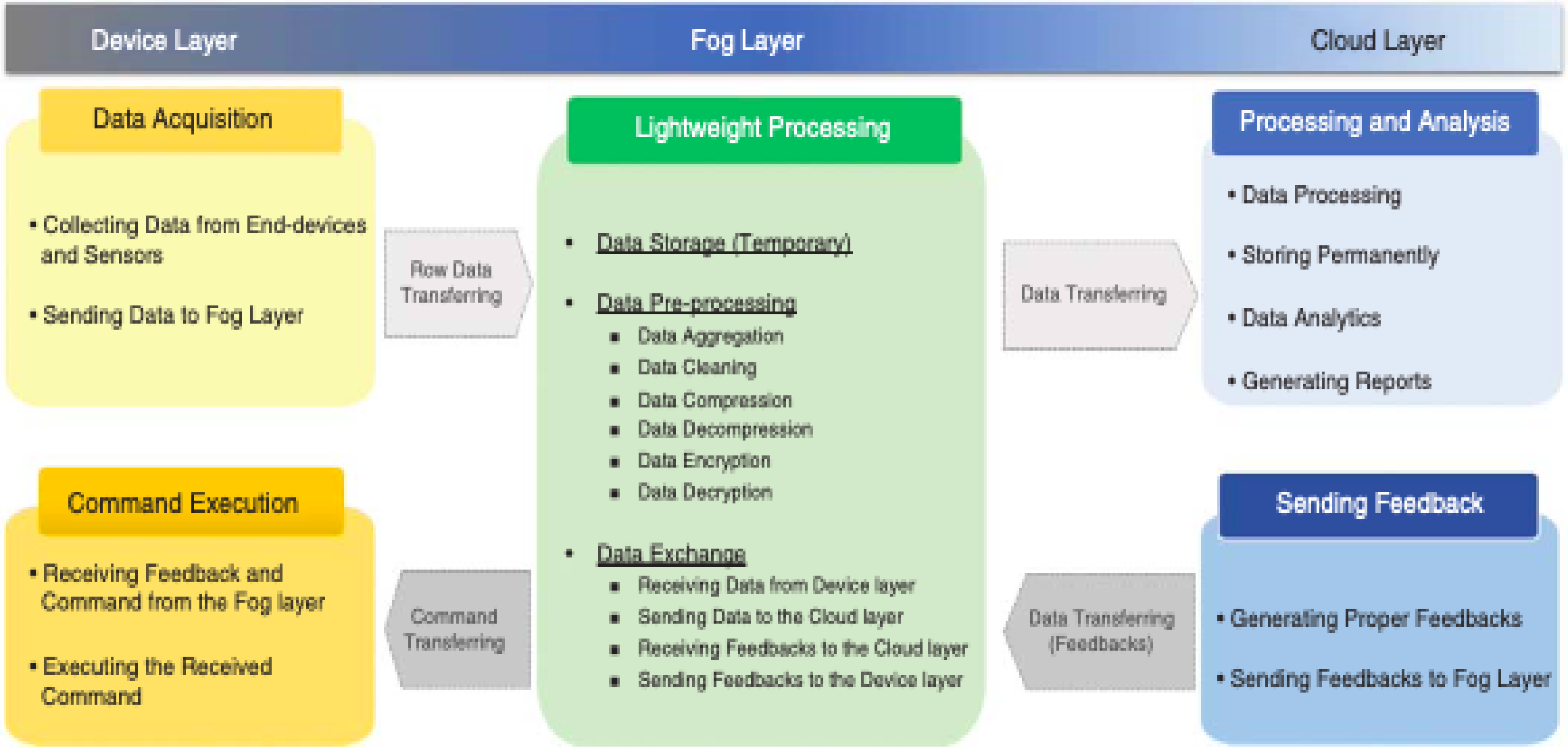
Sending Feedback

Commands or decisions are generated based on processed data and sent to the fog layer.

Command Execution

Actuators execute actions based on feedback received from the fog/cloud layers.

Fog Data Management – IoT Data Characteristics



Fog Data Management – IoT Data Characteristics

Heterogeneity

- End devices generate data in diverse formats and structures.
- Requires standardization and semantic processing for integration.

Inaccuracy

- Sensor data may have uncertainties due to precision errors or misreading.
- Affects data reliability and decision-making.

Weak Semantics

- Raw data often lacks meaningful context or structure.
- Semantic web technologies can enhance interpretability by enriching raw data with metadata.

Fog Data Management – IoT Data Characteristics

Velocity

- High data generation rates and varying sampling frequencies challenge real-time processing capabilities.

Redundancy

- Repetitive data from multiple devices increases storage and processing overhead.

Scalability

- Large numbers of devices and high sampling rates lead to massive data volumes, requiring scalable solutions.

Inconsistency

- Low precision or misreading in sensor data causes inconsistencies in the dataset.

Fog Data Management – Data Preprocessing

- Data pre-processing is a critical step in fog data management to ensure data quality and usability for analytics and decision-making.
- **Data Cleaning**
  - Purpose : To address "dirty data" issues such as missed or unreliable sensor readings.
  - Approaches :
    - **Declarative Data Cleaning**
      - Uses high-level declarative queries (e.g., Continuous Query Language, CQL) to define constraints on sensor values.
      - Example: Extensible Sensor Stream Processing (ESP), a declarative-based framework for pervasive applications.
    - **Model-Based Data Cleaning**
      - Detects anomalies by comparing raw sensor values with inferred values derived from models.
      - Subcategories include:
        - Regression Models : Polynomial regression, Chebyshev regression.
        - Probabilistic Models : Kalman filter.
        - Outlier Detection Models : Techniques to identify inconsistent or abnormal data points.

Fog Data Management – Data Fusion

- Purpose : Eliminates redundancy, ambiguity, and inconsistencies while integrating data from multiple sources.
- **Categories :**
  - Data-Based Model : Focuses on imperfections like impreciseness, incompleteness, vagueness, or uncertainty.
  - Activity-Based Model : Deals with dependency among data.
  - Role-Based Model : Addresses conflicts and diversity in data.
- **Frameworks :**
  - Imperfect, correlated, inconsistent, and disparate data fusion frameworks.
  - Examples: Intelligent Cycle (IC), Joint Directors of Laboratories (JDL).

Fog Data Management – Edge Mining

- **Purpose** : Reduces data volume through local analytics and stream processing at the edge (fog layer).
- **Definition** : Processes sensory data near or at the point of sensing to convert raw signals into contextually relevant information.
- **Techniques :**
  - Linear Spanish Inquisition Protocol (L-SIP) : A lightweight algorithm for local data compression, reducing data size via state estimation.
  - ClassAct and Bare Necessities (BN) : Algorithms that achieve significant reductions in packet transmission (99.6% and 99.98%, respectively).
  - Collaborative Edge Mining : Extends edge mining to further reduce transferred data size.

Fog Data Management – Data Privacy

- Privacy preservation is a key functionality in fog computing to protect sensitive data from unauthorized access. Key aspects include:
- **Authentication Challenges :**
  - Mobility and dynamic network nature (e.g., smart transportation) require adaptive authentication mechanisms.
- **Location Privacy :**
  - Sensitive location data must be protected to prevent misuse.
  - Secure positioning protocols ensure correctness, positioning security, and location privacy.
- **Privacy-Preserving Data Aggregation :**
  - Techniques such as Chinese Remainder Theorem, one-way hash, and homomorphic Paillier encryption enable fault-tolerant aggregation, authentication, and detection of false data injection in the fog layer.

Fog Data Management – Data Storage and Management

- Efficient data storage and placement are essential for minimizing latency and optimizing resource usage in fog computing environments.
  - **Storage Policies :**
    - Data may be discarded after pre-processing or temporarily stored in fog devices for further processing or aggregation.
    - Constraints include storage capacity, memory limitations, and the need for low-latency cache management.
  - **Decision Factors :**
    - Duration and volume of stored data depend on application type and infrastructure capabilities.

Fog Data Management – Data Storage and Management

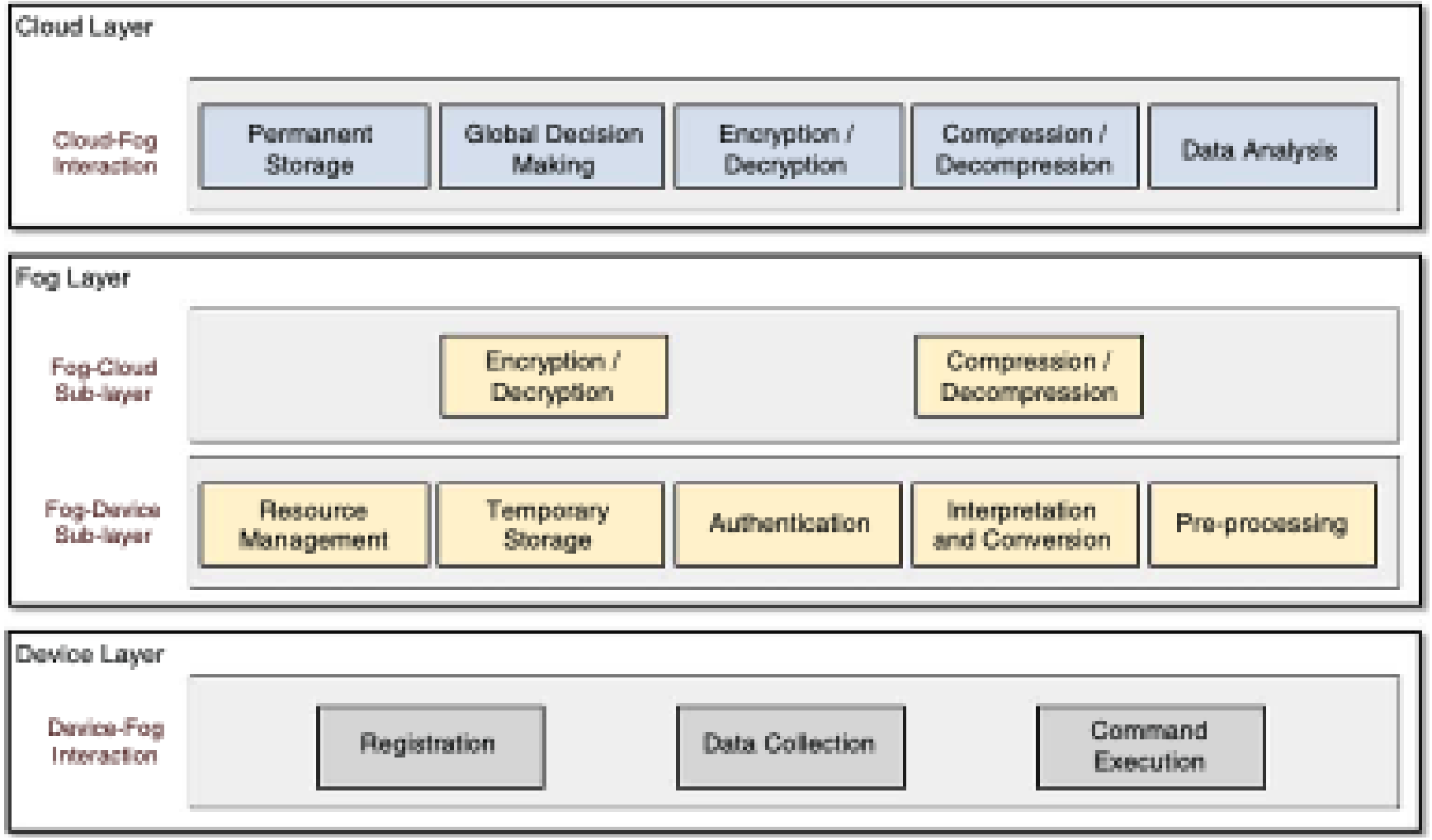
- **Placement Strategies :**
  - Efficient placement of data in fog storages considers node characteristics, geographical zones, and application types.
- **iFogStore :**
  - Proposed by Naas et al., it reduces latency by considering fog device heterogeneity, location, and capacity constraints.
  - Supports dynamic data consumer locations and efficient data sharing.

Fog Data Management – Architecture

- **Data Management Architecture :**
  - A three-layer architecture was proposed in [30] for real-time decision-making:
  - Edge (Fog) Layer : Includes six components—monitoring, data preparation, adaptive algorithm, specification list, storage, and mediator component—for storage-constrained systems.
  - Cloud Layer : Stores historical data.
  - Gathering Layer (Device Layer) : Generates raw data.

Fog Data Management – Architecture

➢Data Management Architecture



Fog Data Management – Architecture

- **Data Management Architecture**
- **Device Layer Modules:**

- Registration Module: Allows devices to dynamically join or leave the network, receiving a unique ID and key for authentication.

- Data Collection/Command Execution Module: Collects data from sensors and executes commands received from the fog layer.

Fog Data Management – Architecture

- **Data Management Architecture**
- **Fog Layer Modules:**

- Resource Management: Manages device registrations and deactivates inactive devices based on policy.
- Temporary Storage: Stores incoming data and device specifications for authentication.
- Pre-Processing: Prepares, aggregates, and processes data for analysis, including data cleaning and filtering.
- Encryption/Decryption and Compression/Decompression: Enhances data privacy and reduces network overhead.

Fog Data Management – Architecture

- **Data Management Architecture**
- **Fog Layer Functions:**

  - Acts as an intermediate layer for data processing and analysis, reducing latency by processing data closer to the source.

  - Supports heterogeneous communication protocols for device interaction.

Fog Data Management – Architecture

- **Data Management Architecture**
- **Cloud Layer Modules:**

- Permanent Storage: Stores large volumes of data from fog zones, utilizing big data technologies.
- Global Decision-Making: Processes data to provide feedback to lower layers and stores useful data.
- Data Analysis: Performs pattern recognition and knowledge discovery from heterogeneous data.

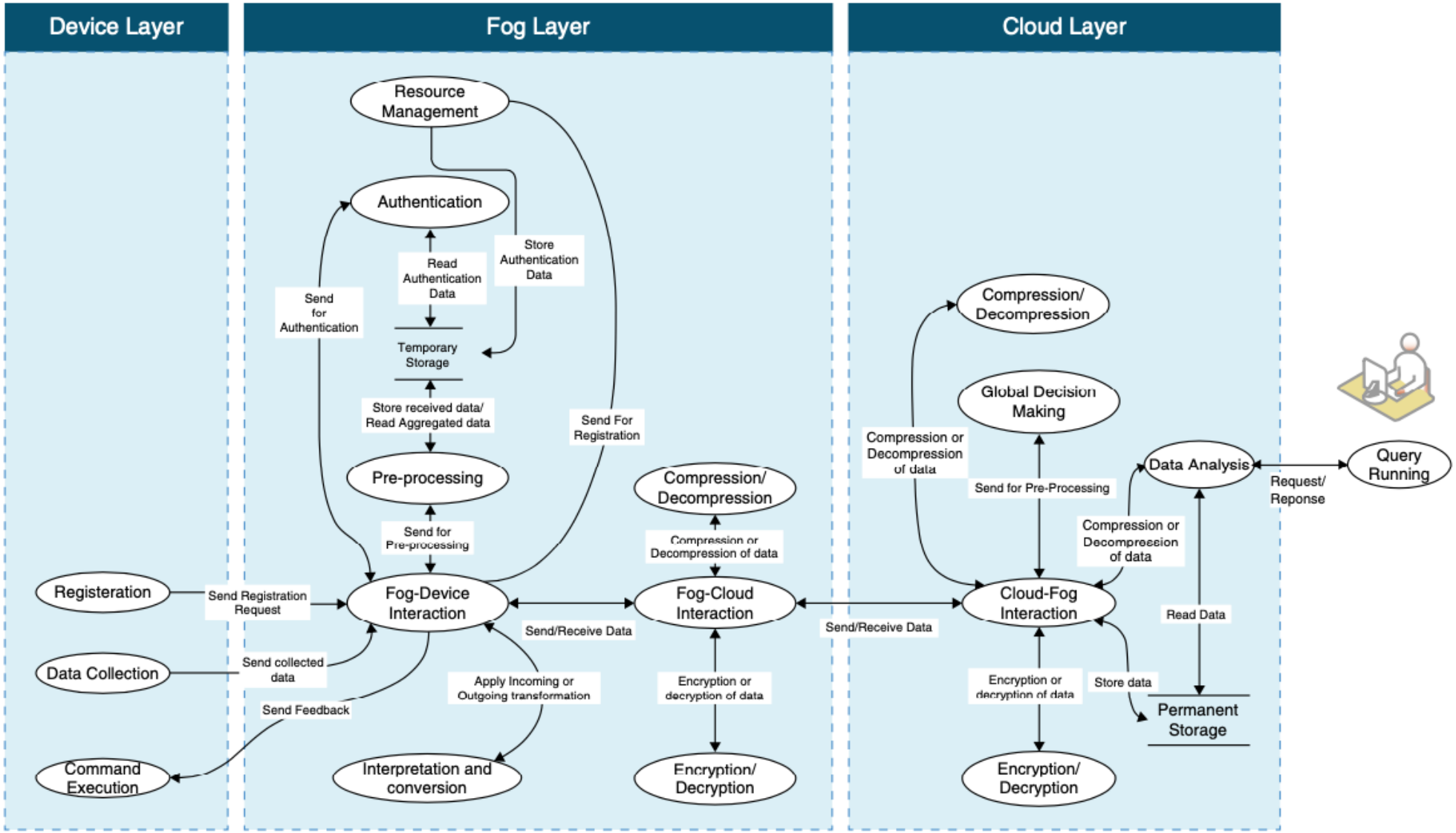
Fog Data Management – Architecture

- **Data Management Architecture**
- **Security and Efficiency:**

- Both fog and cloud layers implement encryption and compression to protect data privacy and reduce network overhead.

- Fog computing enhances security by providing additional measures such as authentication and encryption at the edge.

Fog Data Management – Architecture Interaction



Fog Data Management – Case Study - eHealth

- Fog computing is critical for managing large volumes of data generated by e-Health devices (e.g., ECG devices that produce several GBs of data daily).

- It helps reduce network bandwidth, storage, and processing overhead by handling data locally at the edge (fog layer) rather than sending all data directly to the cloud.

Fog Data Management – Case Study - eHealth

- e-Health systems monitor patients' health parameters (e.g., blood glucose, blood pressure, heartbeat, motion, location) using sensors that transmit data to a local gateway (fog device) at short intervals (e.g., every minute).

- The fog layer performs pre-processing to detect emergency conditions (e.g., blood glucose > 400 mg/dl or blood pressure > 140/90 mmHg).

- In emergencies, immediate actions are taken through fog devices, such as triggering an insulin injection via a bracelet or notifying emergency services.

Fog Data Management – Case Study - eHealth

- e-Health applications provide 24/7 monitoring and control of patients at a lower cost compared to traditional nursing care.

- They offer benefits like ease of use, cost reduction, increased availability, and better services .

Fog Data Management – Case Study - eHealth

- Before sending data to the cloud, the fog layer applies data compression (to reduce size) and encryption (to ensure privacy).

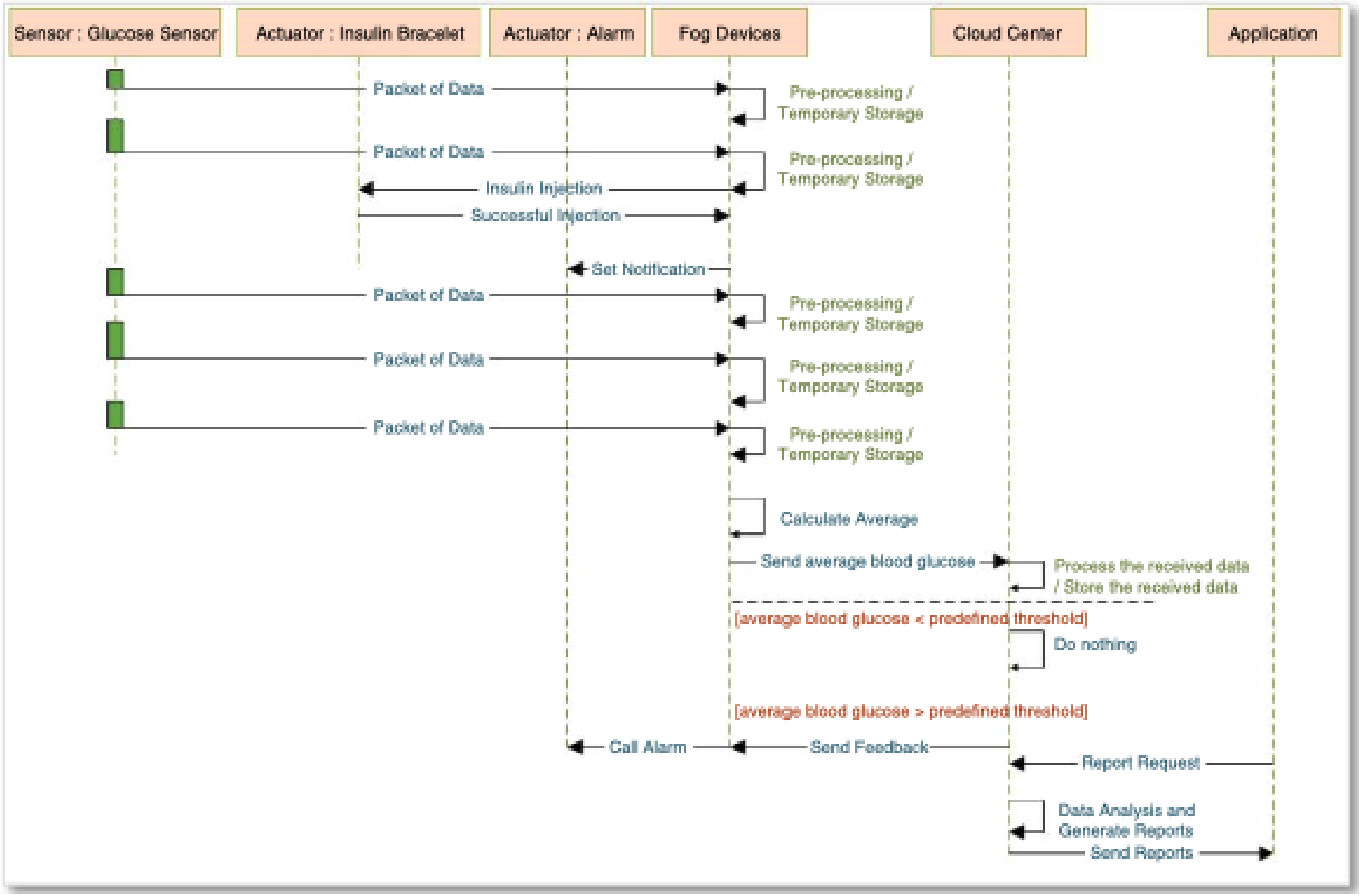
- Preliminary data aggregation, manipulation, and processing occur in the fog layer, ensuring only relevant or processed data is sent to the cloud.

Fog Data Management – Case Study - eHealth

- After initial processing in the fog layer, data is sent to the cloud layer at predefined intervals or based on specific events.

- The cloud stores and processes the data for long-term analysis, enabling users (e.g., doctors, patients) to access health reports and insights.

Fog Data Management – Case Study - eHealth



Predictive Analysis with FogTorchΠ

- FogTorchΠ is an open-source Java prototype designed to determine eligible deployments for multi-component applications on fog infrastructures.

designed to support predictive analysis for fog application deployment

- It considers QoS (Quality of Service) , context , and cost-awareness during deployment decisions.

Predictive Analysis with FogTorchΠ - Inputs

- **Infrastructure (I)**
  - Describes the available infrastructure for application deployment
    - Includes IoT devices , fog nodes , and cloud data centers with their hardware/software capabilities.
    - Specifies probability distributions of QoS metrics like latency and bandwidth for communication links:
      - Cloud-to-fog
      - Fog-to-fog
      - Fog-to-things
  - Includes costs for:
    - Purchasing sensed data from IoT devices.
    - Using virtual instances in cloud/fog environments.

Predictive Analysis with FogTorchΠ - Inputs

- **Multicomponent Application (A):**
  - Specifies the requirements of each application component:
    - Hardware needs : CPU, RAM, storage.
    - Software needs : Operating systems, libraries, frameworks.
    - IoT requirements : Types of IoT devices/things needed.
    - QoS needs : Latency and bandwidth required for interactions between:
      - Component-to-component.
      - Component-to-IoT devices.

Predictive Analysis with FogTorchΠ - Inputs

- **Things Binding (ϑ):**
  - Maps each IoT requirement of an application component to an actual IoT device/thing available in the infrastructure (I).

- **Deployment Policy (δ):**
  - Defines constraints on where application components can be deployed:
    - Based on security or business-related rules .
    - Whitelists eligible nodes (e.g., specific fog nodes or cloud data centers).

Predictive Analysis with FogTorchΠ - Workflow

- **Input:** the inputs (Infrastructure, Application, Things Binding, Deployment Policy) are provided to the tool.

- **Output**: FogTorchΠ generates eligible deployments that satisfy all constraints and requirements.

Predictive Analysis with FogTorchΠ – Workflow - Example

➢ Smart Fan Controlled through Smartphone

Infrastructure	Applications	Things Binding	Deployment Policy
➢ **Devices** • Smartphone (User Interface) • Smart Fan (IoT device), Fog Node (e.g., local gateway). ➢ **Network** • Wi-Fi/Bluetooth - to connect the fan to the fog node and the smartphone.	**Component 1** Fan Control App (on Smartphone) – Sends ON/OFF signals. **Component 2** Control Logic (on Fog Node) – Processes the command and controls the fan.	Which Components on what needs to be found? Bind smartphone input to the fan actuator via the fog node	• Latency: Low (<100 ms for real-time fan control). • Security: Encrypted communication between devices. • Reliability: Ensure continuous operation even if the cloud is unavailable.

Predictive Analysis with FogTorchΠ – Workflow - Example

- Smart Fan Controlled through Smartphone
  - Step 1 – Processing

  - **Input Validation**: FogTorchΠ verifies that the infrastructure, application, and bindings meet the constraints.

  - **Deployment Generation**: It suggests eligible deployments where:
    - Control logic runs on the fog node for low latency.
    - The smartphone interfaces with the fan through the fog node.

Predictive Analysis with FogTorchΠ – Workflow - Example

- Smart Fan Controlled through Smartphone
  - Step 2 – Workflow Execution
  - User Input:
    - The user sends a command (e.g., "Fan ON") via the smartphone app.
  - Data Transfer:
    - The smartphone sends the signal to the fog node over Wi-Fi/Bluetooth.
  - Processing at the Fog Node:
    - The fog node verifies the command and triggers the smart fan.
  - Fan Activation:
    - The fan responds by turning ON or OFF as instructed.
  - Feedback to User:
    - The status (e.g., "Fan is ON") is sent back to the smartphone.

Predictive Analysis with FogTorchΠ – Workflow - Example

- Smart Fan Controlled through Smartphone
  - Step 3 – Output
  - **Valid Deployment:**
    - Smartphone → Fog Node (Processing) → Smart Fan (Actuation)
  - **Constraint Satisfaction:**
    - Low latency, reliable operation, and secure communication.

Predictive Analysis with FogTorchΠ – Workflow - Example

- Smart Fan Controlled through Smartphone

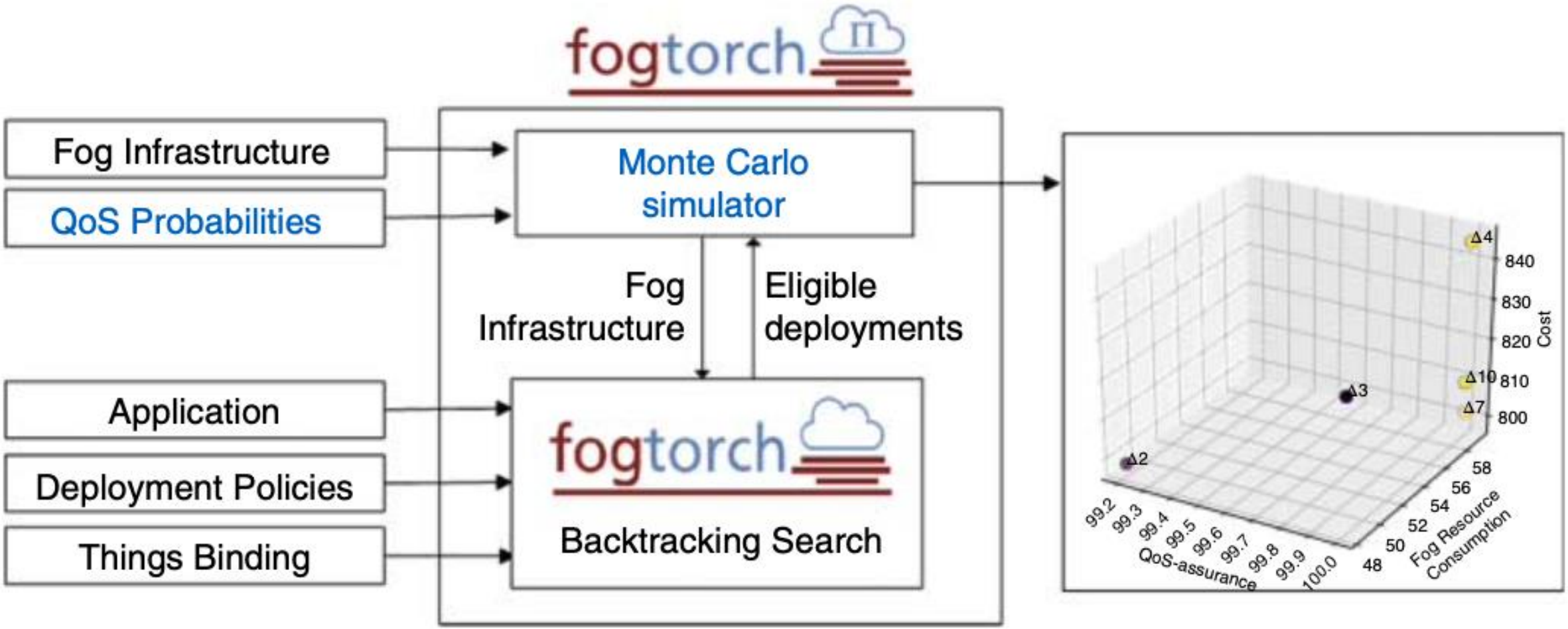
If there were more than one Fog Node, then where will you place the Control Logic

Fan Control Logic

User Fan Controlling App

Actuator Embedded System Firmware

Predictive Analysis with FogTorchΠ - Workflow



FogTorchΠ – What it does?

- FogTorchΠ is a tool that helps figure out the best way to deploy (place) the parts of an application onto cloud or fog nodes.

- It ensures that all requirements are met, such as hardware, software, network speed (latency), and bandwidth.

FogTorchΠ – How Does it Find Eligible Deployments?

- **Eligibility Rules for Deployment:**
  - Each part (component) of the application must be placed on a cloud or fog node that:
    - Follows the deployment rules (e.g., security or business constraints).
    - Meets the hardware and software needs of the component.
    - Can connect to all required IoT devices (directly or through a network link).
    - Has enough resources (like CPU, memory) to handle all components placed on it.
    - Ensures the required speed (latency) and data transfer rate (bandwidth) for communication between components and IoT devices.

FogTorchΠ – How Does it Find Eligible Deployments?

Step 1: Preprocessing

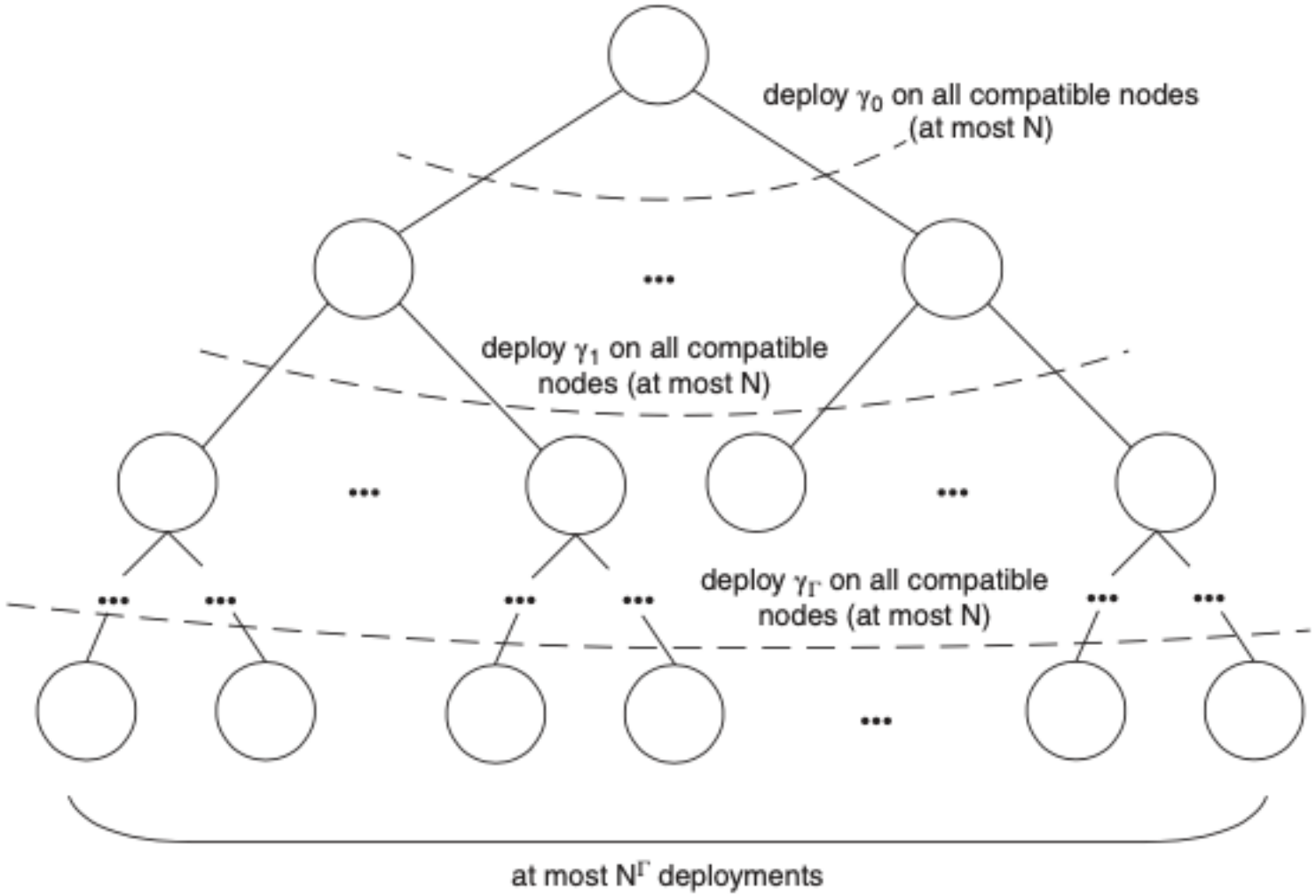
- Before searching for deployments, FogTorchΠ checks which cloud or fog nodes can host each component based on:
- Deployment rules.
  - Hardware/software needs.
  - Latency requirements for connecting to IoT devices.
- If no suitable node is found for even one component, the process stops immediately because there's no valid deployment.

FogTorchΠ – How Does it Find Eligible Deployments?

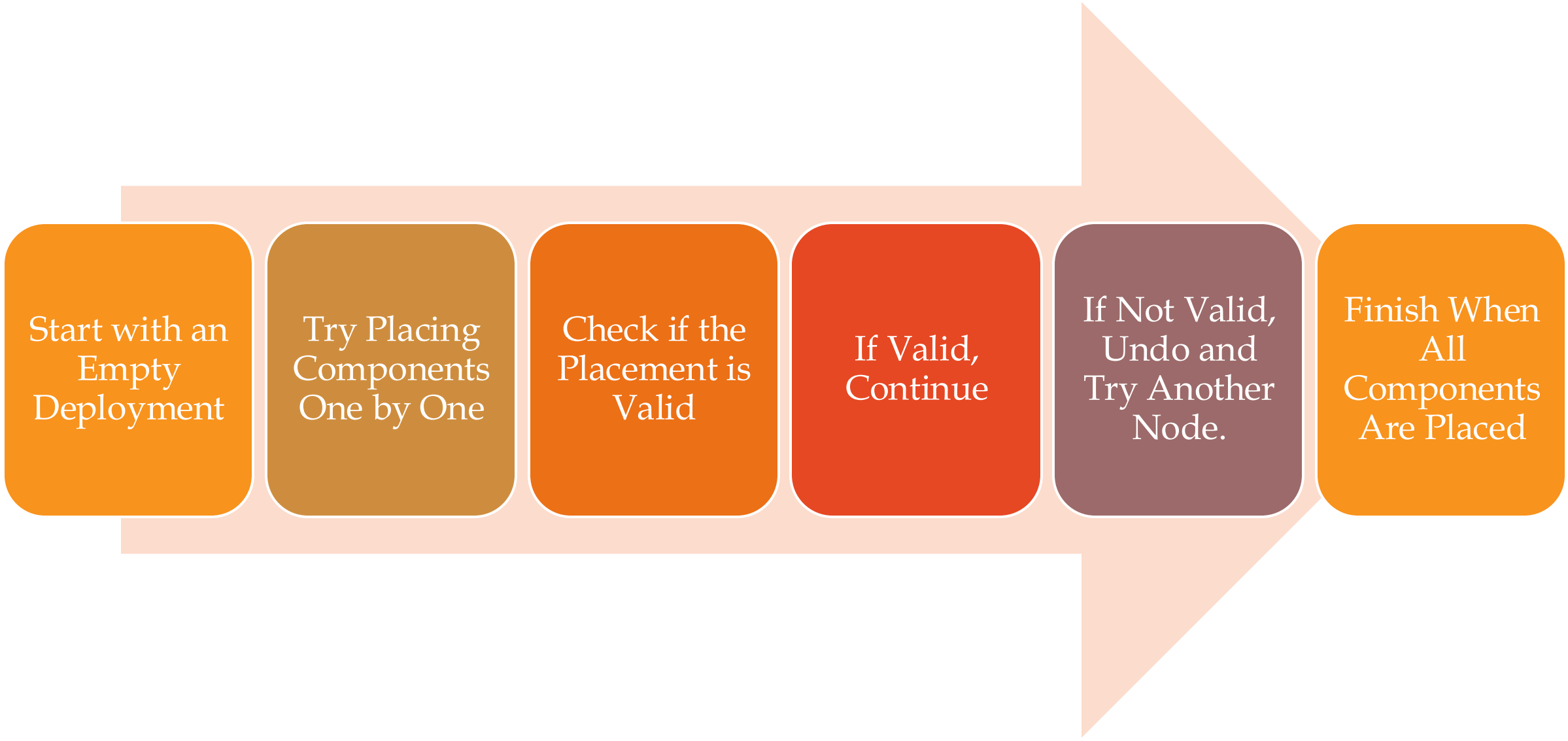
Step 2: Backtracking Search

- FogTorchΠ uses a method called backtracking search to explore all possible ways to deploy the components.
- Imagine a tree where:
  - Each level represents deploying one component.
  - The root (top) is an empty deployment (nothing deployed yet).
  - Leaves (bottom) represent complete deployments where all components are placed.
  - Each branch explores placing a component on a specific node.

FogTorchΠ – How Does it Find Eligible Deployments?



FogTorchΠ – How Does it Find Eligible Deployments?



FogTorchΠ – How Does it Find Eligible Deployments?

Start with an Empty Deployment:

Begin with no components deployed.

Try Placing Components One by One:

Pick a component that hasn't been deployed yet.

Try placing it on a suitable node (from the preprocessed list).

Check if the Placement is Valid:

Ensure the node has enough resources (CPU, memory, bandwidth).

Check if the placement meets latency and bandwidth requirements.

If Valid, Continue:

Update the available resources on the node (subtract what the component uses).

Move to the next component and repeat.

If Not Valid, Undo and Try Another Node:

If the placement doesn't work, "undo" it (release resources) and try another node.

Finish When All Components Are Placed:

Once all components are successfully placed, save the deployment as a valid option.

FogTorchΠ – How Does it Find Eligible Deployments?

```
1:  procedure BACKTRACKSEARCH(D, Δ, A, I, K, ϑ)
2:      if ISCOMPLETE(Δ) then
3:          ADD(Δ, D)
4:          return
5:      end if
6:      γ ← SELECTUNDEPLOYEDCOMPONENT(Δ, A);
7:      for all n ∈ SELECTDEPLOYMENTNODE(K[γ], A) do
8:          if ISELIGIBLE(Δ, γ, n, A, I, ϑ) then
9:              DEPLOY(Δ, γ, n, A, I, ϑ)
10:             BACKTRACKSEARCH(D, Δ, A, I, K, ϑ)
11:             UNDEPLOY(Δ, γ, n, I, A, ϑ)
12:         end if
13:     end for
14: end procedure
```

Figure 9.6 Pseudocode for the backtracking search.

FogTorchΠ – How Does it Find Eligible Deployments?

- **Why Is This Efficient?**

- Preprocessing Saves Time: By filtering out unsuitable nodes early, FogTorchΠ avoids wasting time on bad options.

- Backtracking Explores Options Systematically: It tries all possibilities but backtracks (undoes) when something doesn't work, ensuring no valid deployment is missed.

FogTorchΠ – How Does it Find Eligible Deployments?

- **Time Complexity**

- **Preprocessing:** Takes time proportional to the number of nodes ($O(N)$).

- **Search:** Takes time proportional to the number of nodes raised to the power of the number of components ($O(N^{\Gamma})$), where:
  - N = Number of cloud/fog nodes.
  - Γ = Number of components in the application.

FogTorchΠ – How Does it Estimates?

- how FogTorchΠ estimates:
  - Resource consumption on fog nodes.
  - Monthly costs for deploying an application across cloud and fog infrastructure.

FogTorchΠ – Estimating Fog Resource Consumption

- **What is Measured?**
  - The tool calculates the aggregated average percentage of consumed resources (RAM and storage) across all fog nodes hosting the application.
- **How is it Calculated?**
- For each component of the application:

$$\frac{1}{2}\left(\frac{\sum_{\gamma \in A} RAM(\gamma)}{\sum_{f \in F} RAM(f)} + \frac{\sum_{\gamma \in A} HDD(\gamma)}{\sum_{f \in F} HDD(f)}\right)$$

  - RAM(γ) : RAM required by the component.
  - HDD(γ) : Storage required by the component.
  - RAM(f) : Total RAM available at the node.
  - HDD(f) : Total storage available at the node.

FogTorchΠ – Estimating Monthly Costs

- The cost estimation considers three main categories: hardware , software , and IoT (thing) costs.
- **Hardware Costs**
- Two types of hardware offerings are considered:
  - **Default VMs** : Predefined configurations with fixed monthly fees.
  - **On-demand VMs** : Customizable configurations priced based on individual resources (CPU, RAM, HDD).

FogTorchΠ – Estimating Monthly Costs

- **Cost Calculation**

$$p(H,n) = \begin{cases} c(H,n) & \text{if } H \text{ is a default } VM \\ \sum_{\rho \in R} \left[H.\rho \times c(\rho,n) \right] & \text{if } H \text{ is an on-demand } VM \end{cases}$$

  - If using a default VM:

    Cost = Fixed monthly fee for the VM.

  - If using an on-demand VM

    Cost = Sum of (resource amount × unit cost of the resource) for CPU, RAM, and HDD.

FogTorchΠ – Estimating Monthly Costs

- **Software Costs**
- Two types of software offerings are considered:
  - **Predefined Bundles** : Fixed monthly fees for a set of software features.
  - **On-demand Software** : Individual software components sold separately.

FogTorchΠ – Estimating Monthly Costs

- Cost Calculation
  - If using a bundle:

    Cost = Fixed monthly fee for the bur

$$p(S,n) = \begin{cases} c(S,n) & \text{if S is a bundle} \\ \sum_{s \in S} c(s,n) & \text{if S is on-demand} \end{cases}$$

  - If using on-demand software:

    Cost = Sum of (monthly cost of each software component).

FogTorchΠ – Estimating Monthly Costs

- **IoT (Thing) Costs**
- Two pricing models are considered:
  - **Subscription-based** : Fixed monthly fee for unlimited access to the IoT device.
  - **Pay-per-invocation** : Charged per use of the device (including data transfer costs).

FogTorchΠ – Estimating Monthly Costs

- Cost Calculation

$$p(T,t) = \begin{cases} c(T,t) & \text{if T is subscription based} \\ T.k \times c(t) & \text{if T is pay-per-invocation} \end{cases}$$

  - **If subscription-based:**
    - Cost = Monthly subscription fee.
  - **If pay-per-invocation:**
    - Cost = Number of expected invocations × Cost per invocation.

FogTorchΠ – Combining Costs for Deployment

- **The total monthly cost of a deployment is calculated by summing up:**
  - Hardware costs for all components.
  - Software costs for all components.
  - IoT costs for all required devices.

Total Cost = Sum of (Hardware Cost + Software Cost + IoT Cost) for all components.

$$cost(\Delta, \vartheta, A) = \sum_{\gamma \in A} \left[p\left(\gamma.\overline{H}, \Delta(\gamma)\right) + p\left(\gamma.\overline{\Sigma}, \Delta(\gamma)\right) + \sum_{r \in \gamma.\overline{\theta}} p(r, \vartheta r) \right]$$

FogTorchΠ – Problem with Initial Cost Estimation

- The initial cost model always prefers on-demand VMs and pay-per-invocation IoT if exact matches for requirements aren't available.

- This can lead to overestimating costs , as cheaper options (e.g., smaller VMs or subscription-based IoT) might satisfy the requirements.

FogTorchΠ – Refined Cost Model

- To address overestimation, FogTorchΠ introduces a best-fit lowest-cost policy :
  - **Hardware** : Chooses the cheapest option between:
    - Default VMs (e.g., tiny, small, medium).
    - On-demand VMs built to match the exact requirements.
  - **Software** : Selects the cheapest compatible version of the required software.
  - **IoT** : Compares subscription-based vs. pay-per-invocation costs and picks the cheaper option.

FogTorchΠ – Refined Cost Model

- Hardware
  - pm(Hardware, Node) = Minimum cost of (default VMs or on-demand VMs) that meet requirements.
- Software
  - pm(Software, Node) = Minimum cost of (bundles or on-demand software) that meet requirements.
- IoT
  - pm(Thing, Device) = Minimum cost of (subscription or pay-per-invocation) that meets requirements.

FogTorchΠ – Key Benefits Cost Model

- **Separates VM and Software Costs**: Allows flexibility for both IaaS (Infrastructure as a Service) and PaaS (Platform as a Service) offerings.

- **Extensible**: Can include other types of virtual instances (e.g., containers) beyond VMs.

- **Cost-Aware Matching**: Ensures the cheapest suitable options are chosen for hardware, software, and IoT.

FogTorchΠ – Example

- Suppose a component requires:
  - Hardware : 1 CPU, 1GB RAM, 20GB HDD.
  - Available options at Cloud 2:
    - Default VM: Small instance (€25).
    - On-demand VM: Custom configuration (€30).

- FogTorchΠ selects the small instance (€25) because it satisfies the requirements at a lower cost.

FogTorchΠ – QoS Assurance

- In addition to estimating resource consumption and cost , FogTorchΠ calculates the QoS-assurance of eligible deployments.

- QoS-assurance measures how likely a deployment is to meet all desired Quality of Service (QoS) constraints (e.g., latency, bandwidth) under varying network conditions.

FogTorchΠ – QoS Assurance – How is it Estimated?

- FogTorchΠ uses:
- To find eligible deployments that satisfy QoS requirements.
- Parallel Monte Carlo Simulations : To simulate variations in network conditions and assess the robustness of deployments.

FogTorchΠ – QoS Assurance – Step by Step Process

- Step 1: Initialize a Dictionary
  - An empty thread-safe dictionary D is created to store:
    - Key (Δ) : Represents an eligible deployment.
    - Value (counter) : Tracks how many times the deployment Δ is generated during the simulation.

- Step 2: Divide Simulation Runs Among Threads
  - The total number of Monte Carlo runs (n) is divided among available worker threads (w).
    - Each thread performs nw = [n / w] runs in parallel.
    - Each thread works on its own local copy of the infrastructure (I).

FogTorchΠ – QoS Assurance – Step by Step Process

- **Step 3: Simulate Network Conditions**
  - At the start of each run:
    - Each worker thread samples a state (Is) of the infrastructure based on the probability distributions of QoS metrics (e.g., latency, bandwidth) for communication links.
    - A sampling function is used to generate random QoS values for links, but FogTorchΠ supports arbitrary probability distributions.
- **Step 4: Find Eligible Deployments**
  - The function FindDeployments(A, Is, θ, δ) is called:
    - It performs an exhaustive backtracking search to find all eligible deployments (E) for the application (A) under the sampled infrastructure state (Is).
  - Eligible deployments must satisfy:
    - Processing requirements (hardware/software).
    - QoS requirements (latency, bandwidth).

FogTorchΠ – QoS Assurance – Step by Step Process

- **Step 5: Update Deployment Counts**
  - After each run:
    - The set of eligible deployments (E) is merged into the dictionary D using the function UnionUpdate(D, E) :
    - If a deployment (Δ) is new (not already in D), it is added with a counter of 1.
    - If a deployment (Δ) already exists in D , its counter is incremented.
- **Step 6: Compute QoS-Assurance**
  - After all simulation runs are complete ($n \geq 100{,}000$):
  - For each deployment (Δ) in D :
  - QoS-assurance is calculated as: $\text{QoS-assurance}(\Delta) = \text{counter}(\Delta) / n$

FogTorchΠ – QoS Assurance – Step by Step Process

- This represents the percentage of runs in which the deployment was generated.
- Higher QoS-assurance means the deployment is more likely to meet QoS constraints under varying network conditions.

FogTorchΠ – QoS Assurance – Output

- At the end of the simulation:
  - The dictionary D contains all eligible deployments and their QoS-assurance values.
  - D is returned as the final output.

FogTorchΠ – QoS Assurance – Output

Monte Carlo Simulations	• Allow FogTorchΠ to dynamically simulate changes in network conditions and assess deployment robustness.
Probability Distributions :	• Ensure realistic variations in QoS metrics (e.g., latency, bandwidth) based on historical behavior of the infrastructure.
Parallel Execution :	• Speeds up the simulation by dividing work among multiple threads.
QoS-Assurance Metric :	• Provides a probabilistic measure of how well a deployment will perform in real-world scenarios.

FogTorchΠ – QoS Assurance – Output

- Suppose:
  - Total simulation runs (n) = 100,000.
  - A deployment (Δ) is generated in 85,000 runs.
  - Then:
    - QoS-assurance(Δ) = 85,000 / 100,000 = 0.85 (85%)
  - This means there is an 85% chance that deployment Δ will meet all QoS constraints under varying network conditions.

FogTorchΠ – Problem

Scenario: Deploying a Smart Home Application. Imagine we have a smart home application with three components: Temperature Sensor (IoT device), Data Processor, User Interface. We want to deploy this application in a fog infrastructure consisting of:

- Fog Node 1 (near the home)
- Cloud Node (remote)

- Assumptions:
  - Monthly budget: €500
  - Required QoS-assurance: 99%
  - Temperature sensor generates 10MB data per day
- Cost and resource parameters:
  - Fog Node 1:
    - RAM: 4GB,
    - Storage: 100GB
    - Cost: €200/month
  - Cloud Node:
    - RAM: 8GB, Storage: 500GB
    - Cost: €300/month
    - Data transfer cost: €0.01/MB
  - Component requirements:
    - Data Processor: 2 GB RAM, 50GB storage
    - User Interface: 1GB RAM, 20GB storage

FogTorchΠ – Problem

Resource Consumption calculation:

Resource consumption = (Σ(RAM(component)/RAM(node)) +

Σ(HDD(component)/HDD(node))) / 2

Deployment Option 1: All components on Fog Node 1

Resource consumption = ((2/4 + 1/4) + (50/100 + 20/100)) / 2 = 0.5 or 50%

Deployment Option 2: Data Processor on Cloud, others on Fog

Resource consumption(Fog) = ((1/4) + (20/100)) / 2 = 0.225 or 22.5% Resource

consumption(Cloud) = ((2/8) + (50/500)) / 2 = 0.175 or 17.5%

FogTorchΠ – Problem

Cost Calculation:

Option 1: All on Fog Node 1

Cost = €200 (Fog Node) + (10MB/day * 30 days * €0.01/MB) = €203/month

Option 2: Mixed deployment

Cost = €200 (Fog Node) + €300 (Cloud Node) + (10MB/day * 30 days * €0.01/MB) = €503/month

FogTorchΠ – Problem

Option 1 meets both the budget and resource constraints (€203 < €500, 50% resource usage)

Option 2 exceeds the budget (€503 > €500), though it might provide better performance

FogTorchΠ – Problem 2

Imagine a smart traffic light system with two components:

- Vehicle Sensor (IoT device)
- Traffic Controller

We want to deploy this system in a fog infrastructure consisting of:

- Fog Node 1 (at the intersection)
- Edge Node 2 (nearby lamppost)

- Assumptions:
  - Monthly budget: €200
  - Required QoS-assurance: 98%
  - Vehicle sensor generates 0.5MB data per day
- **Cost and resource parameters:**
  - Fog Node A:
    - RAM: 4GB, Storage: 100GB
    - Cost: €150/month
  - Edge Node B:
    - RAM: 2GB, Storage: 50GB
    - Cost: €100/month
    - Data transfer cost: €0.01/MB
  - **Component requirements:**
    - Traffic Controller: 2GB RAM, 30GB storage

FogTorchΠ – Problem 2

Resource Consumption:

Deployment Option 1: All on Fog Node A

Resource consumption = ((2/4) + (30/100)) / 2 = 0.4 or 40%

Deployment Option 2: Traffic Controller on Edge Node, Vehicle Sensor on Fog Node

Resource consumption(Edge) = ((2/2) + (30/50)) / 2 = 0.8 or 80%

Resource consumption(Fog) = (0 + (0/100)) / 2 = 0%

FogTorchΠ – Problem 2

Cost Calculation

Option 1: All on Fog Node A

Cost = €150 (Fog Node) + (0.5MB/day * 30 days * €0.01/MB) = €150.15/month

Option 2: Mixed deployment

Cost = €150 (Fog Node) + €100 (Edge Node) + (0.5MB/day * 30 days * €0.01/MB) = €250.15/month

FogTorchΠ – Problem 2

Option 1 meets both the budget and resource constraints (€150.15 < €200, 40% resource usage)

Option 2 exceeds the budget (€250.15 > €200), though it might provide better performance

FogTorchΠ – Problem 3

In the Smart Building example discussed in Section 9.4, consider the deployment options Δ2 and Δ16. Using the Monte Carlo simulation method, calculate the QoS-assurance for each deployment option. Assume the following results from the simulation runs:

- Deployment Δ2 was generated in 98,500 out of 100,000 runs .
- Deployment Δ16 was generated in 99,200 out of 100,000 runs .

- Calculate the QoS-assurance percentage for both Δ2 and Δ16.

FogTorchΠ – Problem 3

Step 1: Calculate QoS-Assurance Percentage

The formula for QoS-assurance is derived from the Monte Carlo simulation results:

QoS-assurance (%)=(Number of times deployment is gene rated/Total number of simulation runs)×100

For Δ2 :

QoS-assurance (%)=(98,500/100,000)×100 = 98.5%

For Δ16 :

QoS-assurance (%)=(99,200 / 100,000)×100 = 99.2%

FogTorchΠ – QoS Assurance – Why is QoS-Assurance Important?

- Helps system integrators choose deployments that are robust to network fluctuations.

- Ensures applications meet performance and reliability requirements in real-world scenarios.

Machine Learning in Fog Computing

Fetch Data

- Purpose : Gather raw data from various sources.
- Action : Collect data that is relevant to the problem you are trying to solve.
- Example : Fetching historical sales data, customer behavior data, or sensor readings.

Clean Data

- Purpose : Prepare the data for analysis by removing errors, inconsistencies, or missing values.
- Action : Handle missing values, remove duplicates, correct formatting issues, etc.
- Example : Removing rows with missing sales figures or correcting typos in product names.

Prepare Data

- Purpose : Transform the cleaned data into a format suitable for modeling.
- Action : Perform tasks like feature scaling, encoding categorical variables, splitting data into training and testing sets, etc.
- Example : Converting text data into numerical vectors using techniques like one-hot encoding or TF-IDF.

Machine Learning in Fog Computing

- Purpose : Build a machine learning model using the prepared data.
- Action : Choose a suitable algorithm (supervised or unsupervised) and train it on the training dataset.
  - Supervised Models : Used when you have labeled data (e.g., classification, regression).
    - Examples: Linear Regression, Decision Trees, Neural Networks.
  - Unsupervised Models : Used when you don't have labeled data (e.g., clustering, dimensionality reduction).
    - Examples: K-Means Clustering, PCA (Principal Component Analysis).
- Example : Training a decision tree classifier to predict whether a customer will churn based on their behavior.

Machine Learning in Fog Computing

Deploy the Model

- Purpose : Put the trained model into production so it can make predictions on new data.
- Action : Integrate the model into an application or system where it can be used to make real-time or batch predictions.
- Example : Deploying a fraud detection model in a banking system to flag suspicious transactions.

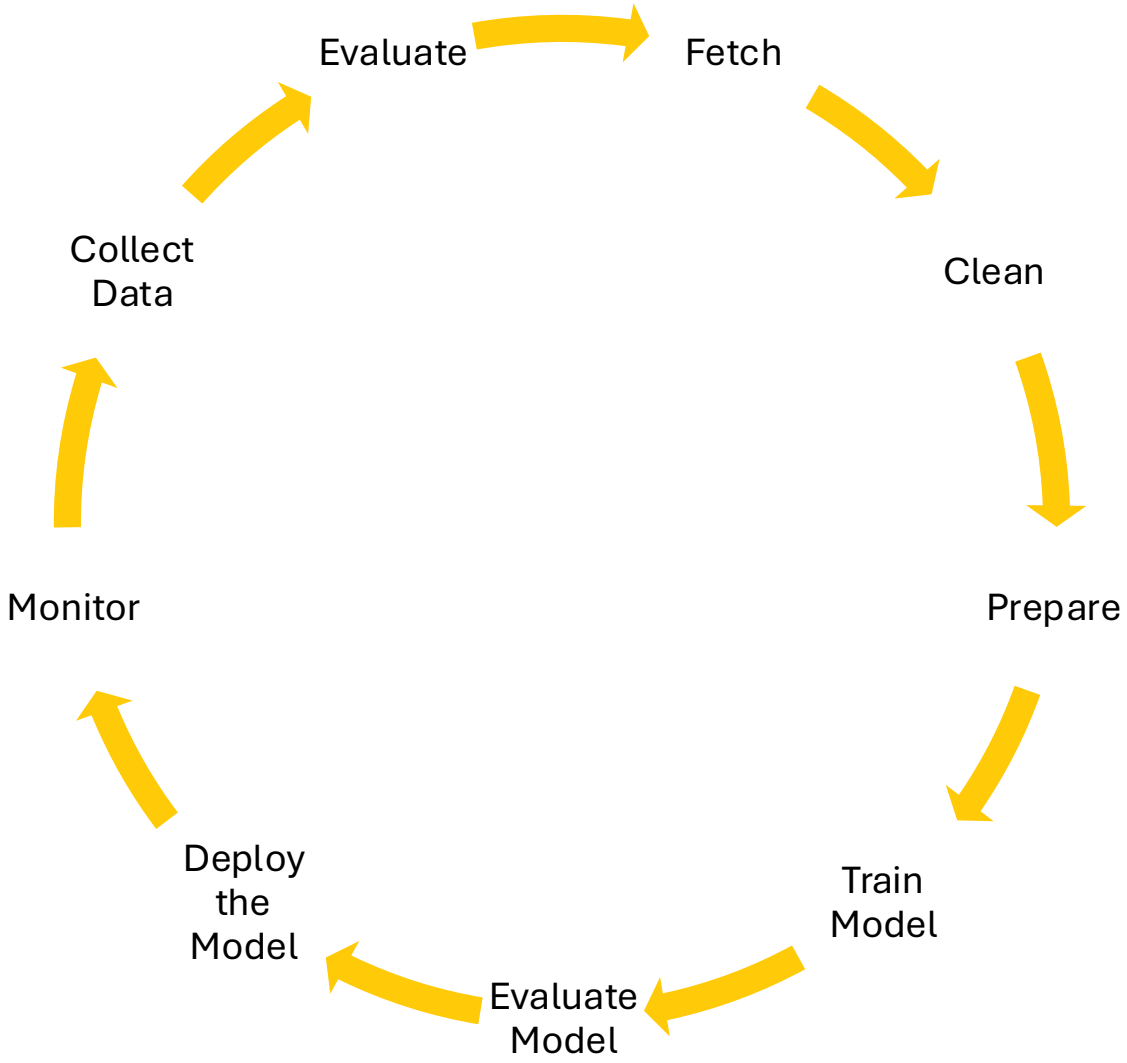
Monitor / Collect Data / Evaluate

- Purpose : Continuously monitor the model's performance and collect new data to improve it over time.
- Action : Track the model's performance in real-world scenarios, gather new data, and retrain the model if necessary.
- Example : Monitoring a recommendation system to see if its accuracy decreases over time and retraining it with new user interactions.

Evaluate Model

- Purpose : Assess the performance of the trained model.
- Action : Use metrics like accuracy, precision, recall, F1-score, or RMSE (for regression) to evaluate how well the model performs on unseen data.
- Example : Checking if the model correctly predicts customer churn with high accuracy.

Machine Learning in Fog Computing



Machine Learning in Fog Computing - Algorithms

Classification

Logistic Regression : Maps predictions between 0 and 1 using the logistic function.

Classification Tree : Splits data into branches to predict output labels.

Support Vector Machine (SVM) : Finds a hyperplane that separates classes with maximum margin.

Naïve Bayes : Uses probability tables and conditional probability for predictions.

K-Nearest Neighbors (KNN) : Predicts class based on the K most similar neighbors in the training data.

Machine Learning in Fog Computing - Algorithms

Regression

Linear Regression : Fits a straight line (or hyperplane) to model relationships.

Regression Tree : Splits data into branches to predict continuous outputs.

K-Nearest Neighbors (KNN) : Predicts values by averaging outputs of K nearest neighbors.

Machine Learning in Fog Computing - Algorithms

Clustering

K-Means : Groups data points into K clusters based on geometric distances; clusters are globular.

DBSCAN : Forms clusters based on density; ignores noise/outliers.

Hierarchical Clustering : Builds a hierarchy of clusters, visualized as a dendrogram.

Machine Learning in Fog Computing - Algorithms

Dimensionality Reduction	PCA (Principal Component Analysis) : Projects data onto a lower-dimensional subspace with maximal variance.
	CCA (Canonical Correlation Analysis) : Finds linear subspaces with high cross-correlation between variables.

Machine Learning in Fog Computing – Algorithms Usage

- **Healthcare**
  - **Metrics to Optimize**: Remote monitoring, disease management, health prediction.
- **Machine-Learning Algorithms:**
  - Classification: Group patients by health condition.
  - Anomaly Detection: Identify potential health issues.
  - Clustering (K-means): Create patient profiles based on similar conditions.
  - Neural Networks: Make real-time decisions for dynamic patient conditions.

Machine Learning in Fog Computing – Algorithms Usage

- **Utilities – Energy/Water/Gas**
  - **Metrics to Optimize**: Usage prediction, demand-supply balance, load balancing.
- **Machine-Learning Algorithms:**
  - Linear Regression: Predict usage for specific times.
  - Classification: Categorize consumers as high, medium, or low usage.
  - Clustering: Group similar users for pattern analysis.
  - Neural Networks: Dynamically balance loads during usage spikes.

Machine Learning in Fog Computing – Algorithms Usage

- **Manufacturing**
  - Metrics to Optimize: Problem diagnosis, failure prediction, security breach detection.
- **Machine-Learning Algorithms:**
  - Decision Trees: Diagnose machine problems.
  - Linear Regression: Predict equipment failure.
  - Anomaly Detection: Detect security breaches or unusual events.

Machine Learning in Fog Computing – Algorithms Usage

- **Insurance**
  - **Metrics to Optimize**: Risk assessment, premium calculation, property damage prediction.
- **Machine-Learning Algorithms:**
  - Clustering (K-means, DBSCAN): Create user profiles based on driving behavior.
  - Classification (Naïve Bayes): Classify customers as risky or safe.
  - Decision Trees: Determine premiums or discounts.
  - Anomaly Detection: Detect theft or property damage.

Machine Learning in Fog Computing – Algorithms Usage

- **Traffic**
- **Metrics to Optimize**: Traffic prediction, bottleneck identification, accident detection.
- **Machine-Learning Algorithms:**
  - DBSCAN: Identify high-traffic roads and intersections.
  - Naïve Bayes: Predict road maintenance needs or accident-prone areas.
  - Decision Trees: Suggest alternative routes to reduce traffic.
  - Anomaly Detection: Detect accidents or unusual traffic patterns.

Machine Learning in Fog Computing – Algorithms Usage

- **Smart City – Citizens and Public Places**
  - **Metrics to Optimize**: Travel patterns, energy consumption, public infrastructure needs.
- **Machine-Learning Algorithms:**
  - DBSCAN: Identify crowded areas at different times.
  - Linear Regression/Naïve Bayes: Forecast energy use or infrastructure needs.
  - CART: Predict passenger travel patterns.
  - Anomaly Detection: Identify abnormal behaviors like terrorism or fraud.
  - PCA: Simplify data analysis by reducing dimensions.

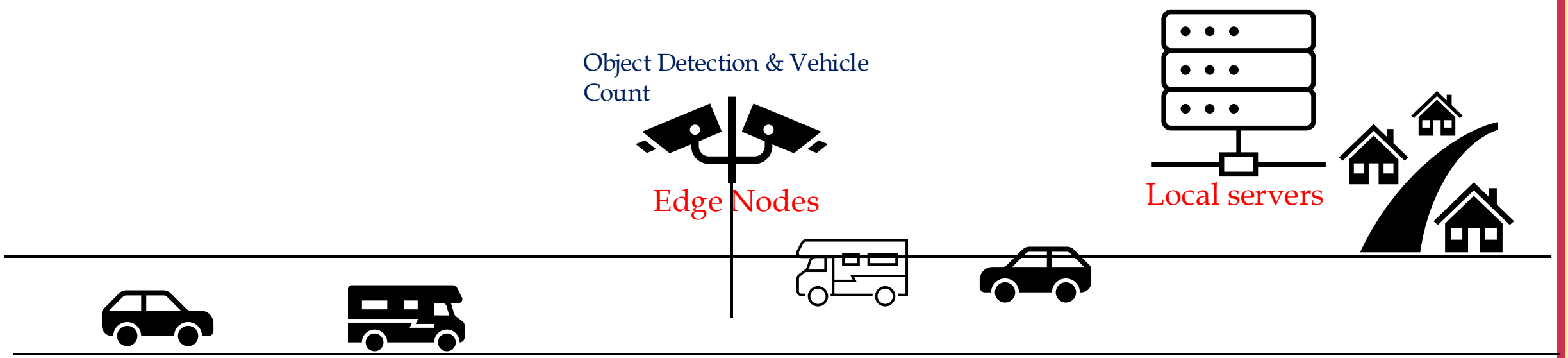
Machine Learning in Fog Computing – Algorithms Usage

- **Smart Homes**
  - **Metrics to Optimize**: Energy consumption, occupancy awareness, intrusion detection.
- **Machine-Learning Algorithms:**
  - K-means: Analyze energy load and consumption patterns.
  - Linear Regression/Naïve Bayes: Predict energy use or occupancy.
  - Anomaly Detection: Detect intrusions, device tampering, or malfunctions.

Machine Learning in Fog Computing – Algorithms Usage

- **Agriculture**
  - **Metrics to Optimize**: Resource optimization, disease detection, crop production prediction.
- **Machine-Learning Algorithms**:
  - Naïve Bayes: Assess crop health.
  - Anomaly Detection: Detect water leakage or uneven supply.
  - Neural Networks: Analyze drone images for weed growth or uneven crop growth.

Machine Learning in Fog Computing

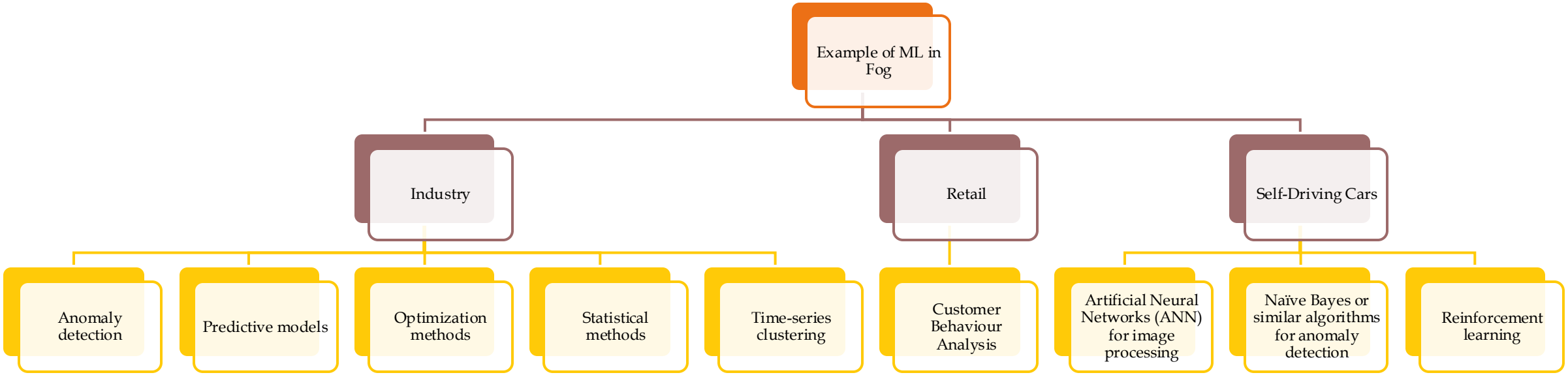


Machine Learning in Fog Computing

Data Centre	• Store historical traffic data • Train advanced predictive models • Forecast trends like peak traffic hours or future infrastructure needs • ML Models for Broader Geographic area	Cloud
Local servers	• Aggregate data from multiple cameras and IoT sensors • Equipped with ML models • Vehicle classification • Traffic density prediction • Identifying anomalies - Unusual Congestion or Accidents • Make quick decisions • optimizing traffic signals • Sending alerts to authorities	Fog Nodes
Cameras and IoT sensors	• Basic pre-processing tasks - motion detection, initial filtering of the footage, reduce data size • Capture real-time video footage and traffic data. • Object Detection & Vehicle Count • Adjusting traffic lights in real-time to manage congestion • Sending alerts to edge displays or mobile apps about accidents.	Edge Nodes

Machine Learning in Fog Computing

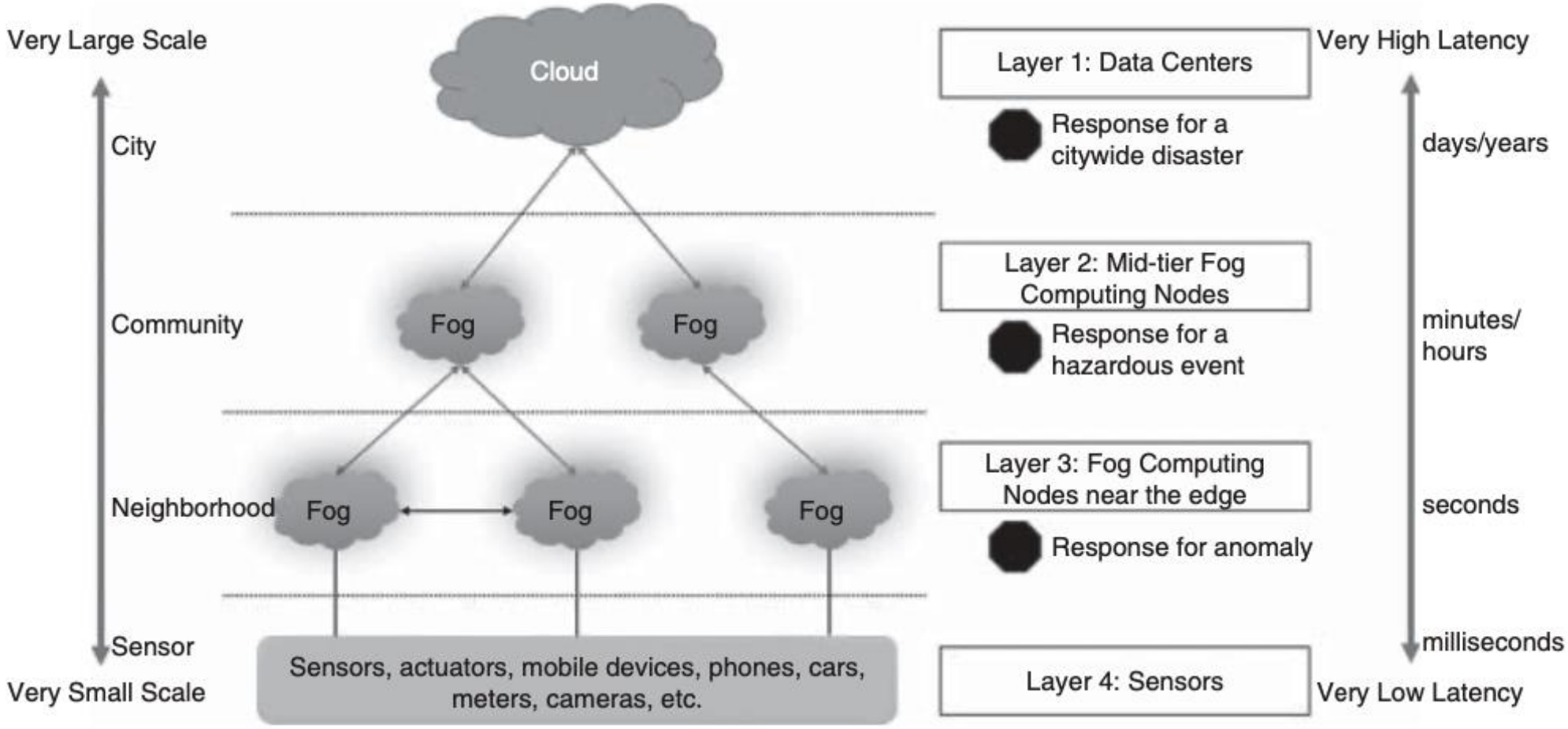
➢Examples of Machine Learning in Fog Computing



Machine Learning in Fog Computing

➢Machine Learning in Fog Computing Security

Hierarchical Fog Architecture for Security



Machine Learning in Fog Computing

➢Machine Learning in Fog Computing Security

Machine-learning algorithms at different fog layers

Layer	Disaster response	ML algorithm
Layer 4 - Sensors	None	None
Layer 3 – Fog nodes for the neighborhood	Response for anomaly	KNN, Naïve Bayes, random forest, DBSCAN
Layer 2 – Fog nodes for the community	Response for hazardous event	HMM, MAP [58] Regression, ANN, decision trees
Layer 1 – Cloud	Response for city-wide disaster, long-term forecasting	ANN, Deep learning, decision trees, reinforcement learning, Bayesian networks

Data Analytics in Fog Computing

- Fog computing is a virtualized platform that provides computation, storage, and networking services between end devices and traditional cloud data centers, typically located at the edge of the network.

- Key Characteristics:
  - Low latency
  - Mobility support
  - Real-time interactions
  - Online analytics
  - Interplay with the cloud

Data Analytics in Fog Computing

➢ Challenges of Cloud-Only Data Processing

Network Infrastructure Stress	• Transferring all IoT-generated data to the cloud for processing or storage is expensive, technically impractical, and often unnecessary.
Latency Issues	• Cloud-based analytics works well for historical data but fails for real-time, high-data-rate applications.
Cost and Storage	• Fog computing reduces communication time, costs, and the need for massive cloud storage by preprocessing data locally.

Data Analytics in Fog Computing

➢ Fog Computing's Role in IoT

Data Preprocessing	• Filters, processes, and summarizes data before sending it to the cloud.
Hybrid Approach	• Combines fog localization (low latency, context-aware) with cloud globalization (centralized storage, historical analysis).
Applications	• Ideal for IoT use cases requiring real-time analytics, such as smart cities, healthcare, and intelligent transportation systems.

Data Analytics in Fog Computing

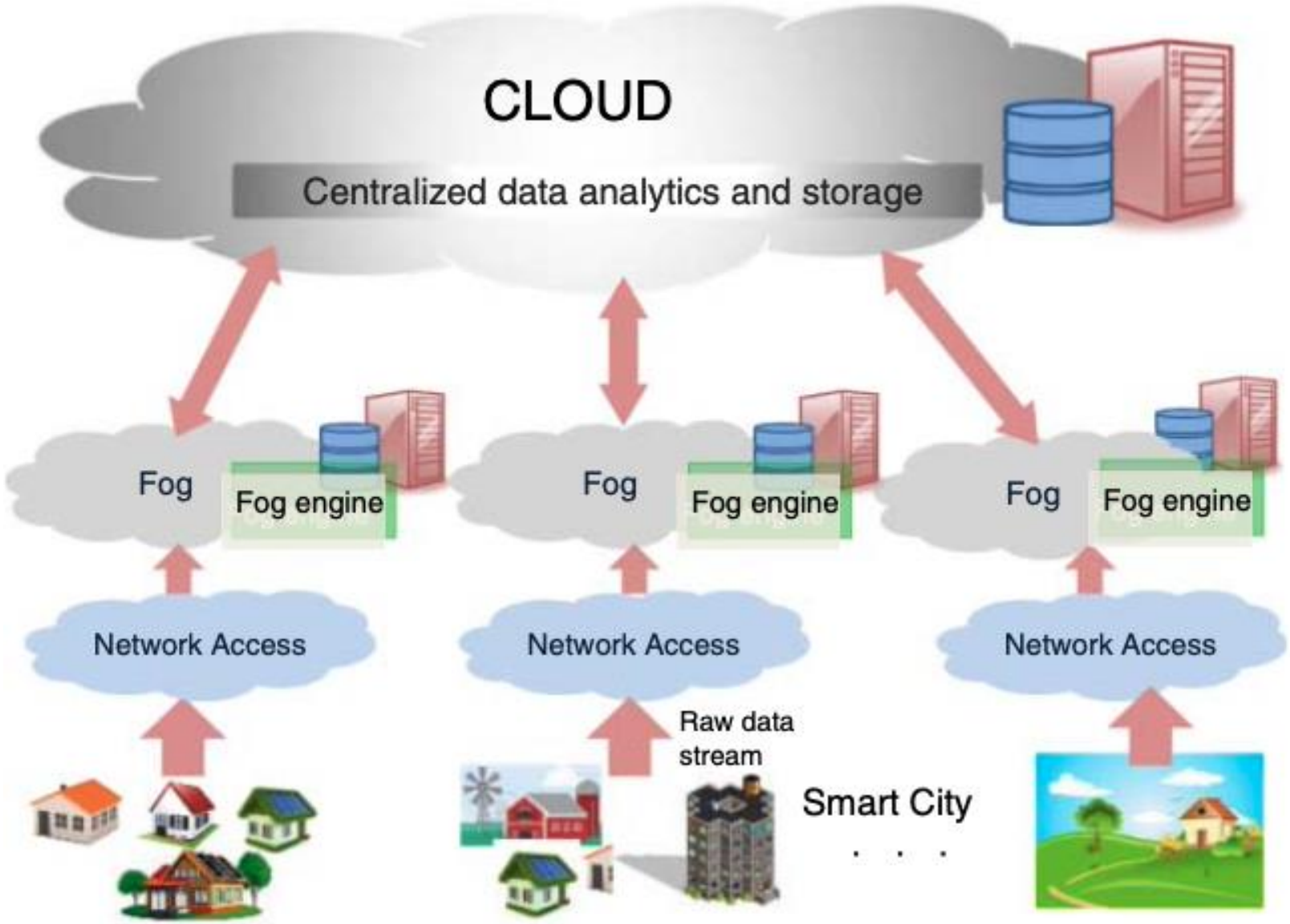
- **Fog Analytics**
  - Localized Processing: Performs preliminary analysis near the data source to reduce the volume of data sent to the cloud.
    - Example: Google Photos uses machine learning on mobile devices (via Movidius chips) for real-time photo classification.
  - Streaming Analytics: Handles continuous incoming data streams for real-time decision-making.
  - Standardization Needs: Requires standardized device/data interfaces, seamless cloud integration, and flexible network architectures.
  - Machine Learning: Enables performance improvements over time for IoT applications.
  - Data Visualization: Provides insights closer to the edge for faster decision-making.

Data Analytics in Fog Computing

- **Fog Engines**
  - Fog-engines (FEs) are customizable, agile platforms integrated into IoT devices for on-premise, real-time data analytics.
  - Transparently managed by end users, FEs provide local peer-to-peer networks beneath the cloud.
  - Capabilities:
    - On-premise and real-time analytics.
    - Communication between IoT devices and the cloud.
    - Offloading data to the cloud for global analytics.

Data Analytics in Fog Computing

- **Fog Engines**

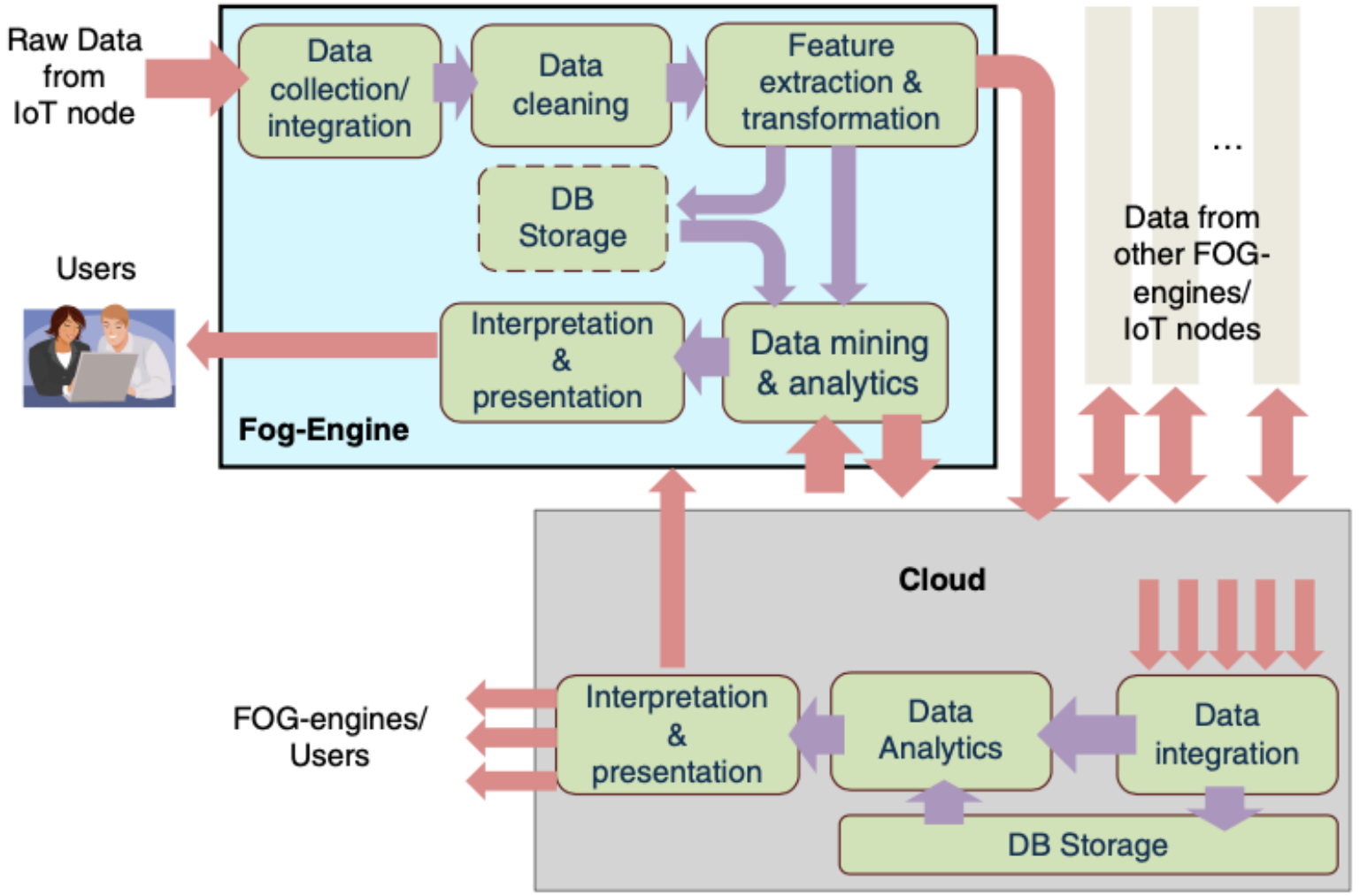


Data Analytics in Fog Computing

- **Fog Engines - Key Features**
  - Modular APIs: Provide consistent functionality across fog-engines and cloud platforms.
  - Hardware Diversity: Utilizes multi-core processors, FPGAs, or GPUs for fine-grained processing.
  - Fault Tolerance: Tasks can be transferred to nearby fog-engines in case of failure.
  - Energy Efficiency: Battery-powered FEs are suitable for IoT applications with limited power access.

Data Analytics in Fog Computing

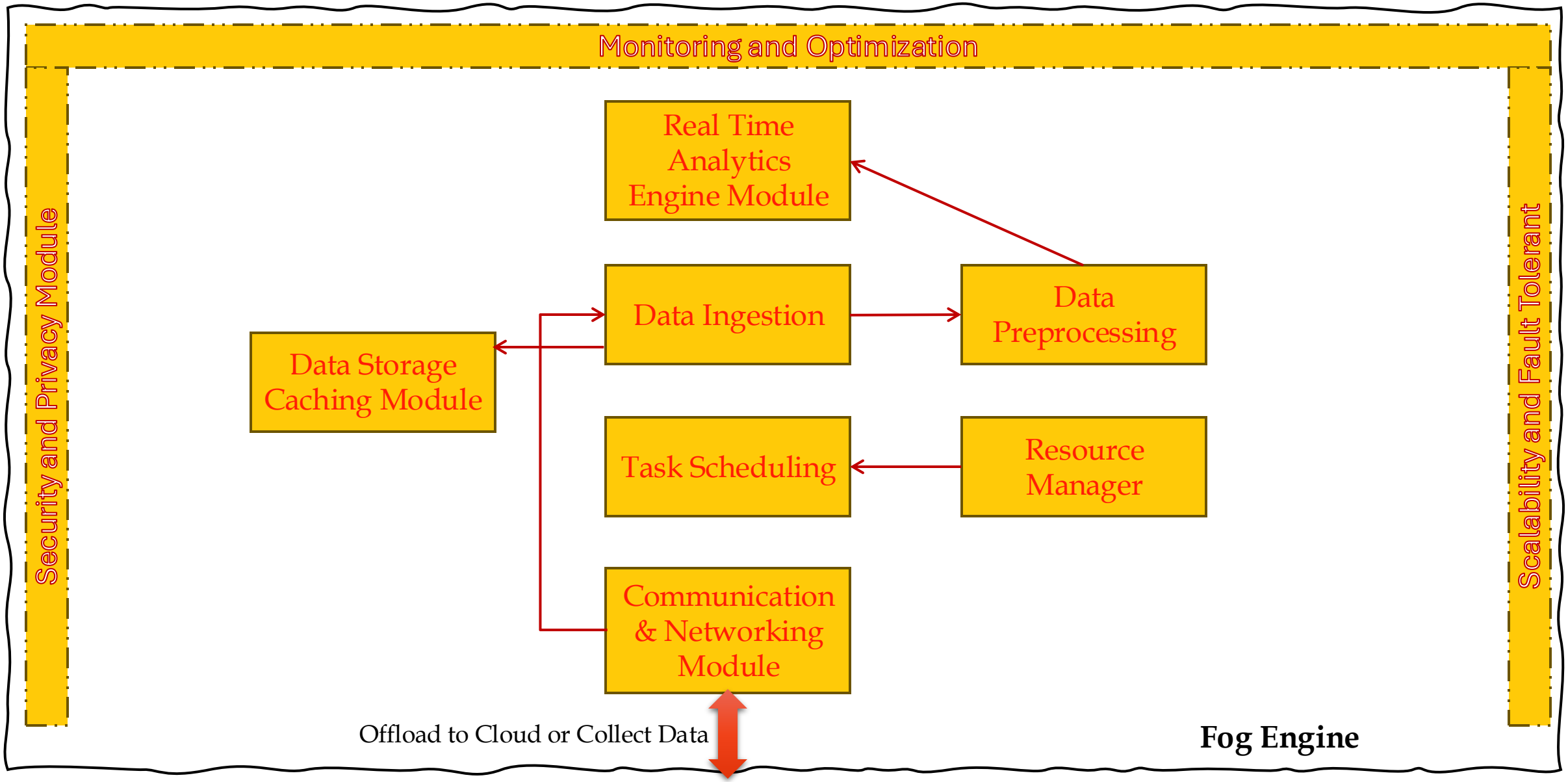
- **Data analytics using a fog-engine before offloading to the cloud**



Data Analytics in Fog Computing – Fog Engine

- A comprehensive architecture designed to support fog computing for data analytics
- Fog computing is an extension of cloud computing that brings computation, storage, and networking closer to the edge of the network (near the data source) to reduce latency, bandwidth usage, and improve real-time decision-making.
- Fog Engine Framework specifically focuses on enabling efficient and scalable data analytics in fog environments by leveraging distributed resources.

Data Analytics in Fog Computing – Fog Engine Components



Data Analytics in Fog Computing - Fog Engine Components

Data Ingestion	• Handles the collection of raw data from various IoT devices and sensors.
Data Preprocessing	• Cleans, filters, and transforms raw data into a structured format suitable for analysis.
Resource Manager	• Manages computational, storage, and networking resources across fog nodes and cloud servers.
Task Scheduling	• Assigns tasks to appropriate fog nodes or cloud servers based on factors like latency, resource availability, and task priority.
Real-Time Analytics Engine	• Performs real-time data processing and analytics using techniques like stream processing, machine learning, and AI.

Data Analytics in Fog Computing - Fog Engine Components

Data Storage & Caching Module	• Stores processed data locally at the fog layer and caches frequently accessed data to reduce latency.
Communication & Networking Layer	• Facilitates seamless communication between edge devices, fog nodes, and cloud servers.
Security & Privacy Module	• Implements encryption, authentication, and privacy-preserving mechanisms to protect data and applications.
Monitoring & Optimization	• Monitors system performance and applies optimization techniques to improve efficiency.
Scalability & Fault Tolerance	• Ensures the system can scale to handle increasing workloads and remains operational during node failures.

Data Analytics in Fog Computing

➢Fog Engines Vs Cloud Computing

Feature	Fog Engine	Cloud Computing
Proximity to Data Source	Near the edge (low latency)	Centralized (higher latency)
Processing Power	Limited	High
Storage Capacity	Limited	Massive
Resource Control	User-configurable	Out of user's control
Energy Efficiency	Suitable for battery-powered IoT devices	Constant power supply
Cost Model	Owned by the user	Pay-as-you-go

Data Analytics in Fog Computing

➢Fog Engines Vs Cloud Computing

Characteristic	Fog-engine	Cloud
Processing hierarchy	Local data analytics	Global data analytics
Processing fashion	In-stream processing	Batch processing
Computing power	GFLOPS	TFLOPS
Network Latency	Milliseconds	Seconds
Data storage	Gigabytes	Infinite
Data lifetime	Hours/Days	Infinite
Fault-tolerance	High	High
Processing resources and granularity	Heterogeneous (e.g., CPU, FPGA, GPU) and Fine-grained	Homogeneous (Data center) and coarse-grained
Versatility	Only exists on demand	Intangible servers
Provisioning	Limited by the number of fog-engines in the vicinity	Infinite, with latency
Mobility of nodes	Maybe mobile (e.g. in the car)	None
Cost Model	Pay once	Pay-as-you-go
Power model	Battery-powered/ Electricity	Electricity

Data Analytics in Fog Computing

- **Data Analytics Using Fog-Engines**
- **Workflow:**
  - Local Analysis: In-stream data is analyzed in real-time by the fog-engine near the data source.
  - Data Offloading: Filtered, preprocessed data is transmitted to the cloud for offline, global analytics.
    - Example: In a smart grid, fog-engines help users optimize energy usage locally, while cloud analytics determines policies for an entire town.

Data Analytics in Fog Computing

- **Advantages:**
- Reduced Data Volume: Preprocessing minimizes the amount of data sent to the cloud.
- Real-Time vs. Offline Analytics: Fog-engines handle real-time analytics; the cloud performs historical analysis.
- Dynamic Fog Networks: Multiple fog-engines in proximity dynamically form a local fog network for collaborative processing.

Data Analytics in Fog Computing

- **Challenges:**
- Architecture Design: Developing scalable, hierarchical fog architectures.
- Frameworks and Standards: Standardizing device/data interfaces and integrating fog with the cloud.
- Resource Management: Efficient provisioning and scheduling of storage and networking resources.
- Security and Privacy: Addressing vulnerabilities in distributed fog environments.